

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
LẬP TRÌNH MẠNG

TRUYỀN VIDEO THỜI GIAN
THỰC
QUA GIAO THỨC IP MULTICAST

(Multicast Video Streaming)

Giảng viên hướng dẫn:	Nguyễn Thanh Quân
Lớp:	23_4
Môn học:	Lập trình mạng
Khoa:	Công nghệ Thông tin

TP. Hồ Chí Minh, tháng 7 năm 2026

Thông tin nhóm và phân công công việc

1. Thông tin thành viên

softblue MSSV	Họ và tên	Vai trò
23120332	Nguyễn Trọng Phúc	Server-side Developer
23120340	Phạm Minh Quân	Client-side Developer

2. Bảng phân công công việc

softblue Thành viên	Nhiệm vụ phụ trách	Mức độ
Nguyễn Trọng Phúc (23120332)	Thiết kế và xây dựng <code>Server.py</code> : đọc file MJPEG (lớp <code>MJPEGReader</code>), tự phát hiện định dạng file, phát lại video khi hết file	100%
	Cài đặt cơ chế đóng gói (packetize) khung hình, phân mảnh (fragmentation) gói tin lớn và kiểm soát tốc độ phát 20 FPS	
	Cấu hình socket UDP Multicast, thiết lập TTL, log thống kê phía Server	
	Viết báo cáo mục: Kiến trúc hệ thống, Định dạng gói tin, Triển khai Server	
	Quay video/ghi lại minh chứng chạy thử Server	
Phạm Minh Quân (23120340)	Thiết kế và xây dựng <code>Client.py</code> : giao diện đồ họa (Tkinter/Pillow), luồng mạng nhận gói tin đa luồng (multi-threading)	100%

	Cài đặt thuật toán ráp lại khung hình từ các mảnh gói tin (reassembly) và tham gia/rời nhóm Multicast	
	Xây dựng module đo lường thông kê: tỉ lệ mất gói, FPS thực tế, băng thông, độ trễ	
	Viết báo cáo mục: Triển khai Client, Thuật toán đo lường, Kết quả kiểm thử	
	Thiết kế bảng đối chiếu tiêu chí chấm điểm	
Cả hai thành viên: cùng thảo luận thiết kế định dạng gói tin (custom packet header), kiểm thử hệ thống với nhiều Client đồng thời, rà soát và hoàn thiện báo cáo tổng thể.		100%

Mục lục

Thông tin nhóm và phân công công việc	i
1 Giới thiệu đề tài	1
1.1 Mô tả bài toán	1
1.2 Yêu cầu chức năng	1
1.2.1 Phía Server	1
1.2.2 Phía Client	1
1.3 Tiêu chí đánh giá của đề bài	2
2 Kiến trúc hệ thống	3
2.1 Mô hình tổng quan	3
2.2 Thông số Multicast sử dụng	3
2.3 Luồng hoạt động	4
3 Định dạng gói tin tùy biến (Custom Packet Format)	5
3.1 Vấn đề phân mảnh khung hình	5
3.2 Cấu trúc Header (24 bytes)	5
4 Triển khai chi tiết	7
4.1 Server (Server.py)	7
4.1.1 Đọc file MJPEG – lớp MJPEGReader	7
4.1.2 Phát lại video tự động (Loop)	7
4.1.3 Kiểm soát tốc độ khung hình (20 FPS)	7
4.1.4 Cấu hình socket Multicast	8
4.2 Client (Client.py)	8
4.2.1 Giao diện người dùng (GUI)	8
4.2.2 Xử lý đa luồng (Multi-threading)	8
4.2.3 Ráp lại khung hình từ các mảnh (Reassembly)	8
4.2.4 Tham gia và rời nhóm Multicast	8
4.2.5 Cơ chế phát hiện trạng thái luồng (Online/Offline)	9
5 Thuật toán đo lường thống kê và phát hiện mất gói	10
5.1 Phát hiện và tính tỉ lệ mất gói (Packet Loss)	10

5.2	Đo tốc độ khung hình thực tế (Actual FPS)	10
5.3	Đo băng thông tiêu thụ (Bitrate)	10
5.4	Đo độ trễ truyền dẫn (Estimated Latency)	11
6	Hướng dẫn cài đặt và chạy chương trình	12
6.1	Yêu cầu môi trường	12
6.2	Khởi chạy Server	12
6.3	Khởi chạy Client	12
7	Kết quả kiểm thử	13
7.1	Kịch bản kiểm thử	13
7.2	Nhận xét	13
7.3	Đối chiếu với tiêu chí chấm điểm	13
8	Kết luận và hướng phát triển	15
8.1	Kết luận	15
8.2	Hướng phát triển	15

Chương 1

Giới thiệu đề tài

1.1 Mô tả bài toán

Đề án yêu cầu xây dựng một ứng dụng truyền phát video (video streaming) theo thời gian thực, sử dụng giao thức **IP Multicast** trên nền tảng UDP. Hệ thống gồm hai thành phần chính:

- **Server:** đọc video định dạng MJPEG (Motion JPEG) theo từng khung hình (frame), đóng gói và quảng bá (broadcast) liên tục tới một địa chỉ nhóm Multicast.
- **Client:** gia nhập nhóm Multicast, nhận các gói tin video, giải mã và hiển thị video theo thời gian thực.

Lưu ý cấu trúc thiết kế từ đề bài: Hệ thống **không** sử dụng RTSP, RTP, không cần lựa chọn giữa TCP/UDP (chỉ dùng UDP) và không tách kênh điều khiển/kênh dữ liệu riêng biệt – toàn bộ được truyền trên một kênh Multicast duy nhất.

1.2 Yêu cầu chức năng

1.2.1 Phía Server

- Đọc file video MJPEG theo từng khung hình.
- Đóng gói (packetize) mỗi khung hình.
- Gửi từng khung hình tới địa chỉ IP Multicast.
- Phát với tốc độ xấp xỉ 20 FPS (50 ms/khung hình).
- Tiếp tục phát cho đến khi hết video, sau đó tự động phát lại từ đầu.

1.2.2 Phía Client

- Gia nhập nhóm Multicast.

- Nhận các gói tin Multicast.
- Giải mã gói tin nhận được.
- Hiển thị video theo thời gian thực.
- Rời nhóm Multicast khi thoát chương trình.

1.3 Tiêu chí đánh giá của đề bài

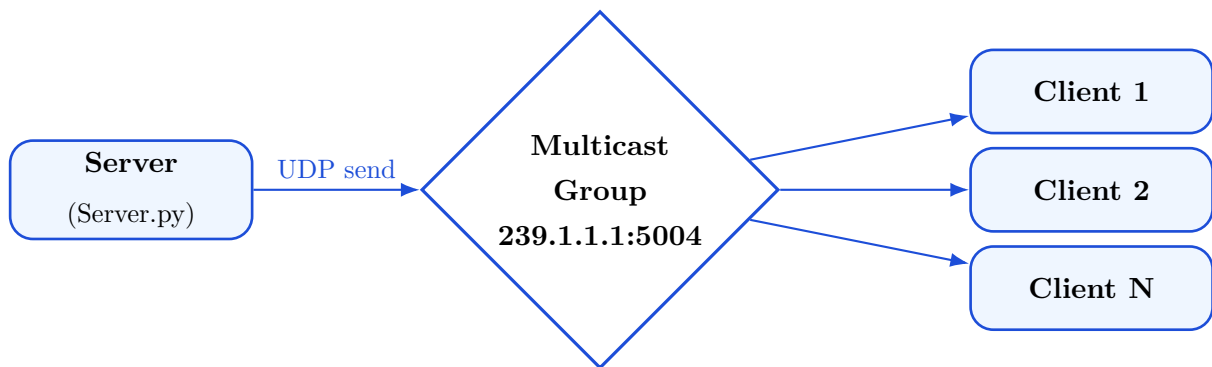
Tiêu chí	Điểm	Mô tả
Cài đặt Server	2.5	Multicast server
Cài đặt Client	2.5	Nhận và hiển thị video
Định dạng gói tin	2.0	Gói tin tùy biến (custom packet)
Nhiều Client & phát hiện mất gói	2.0	Xử lý đồng thời / thống kê
Báo cáo	1.0	Kiến trúc / kiểm thử
softblue Tổng	10.0	

Chương 2

Kiến trúc hệ thống

2.1 Mô hình tổng quan

Hệ thống được thiết kế theo mô hình **One-to-Many** (một Server – nhiều Client) dựa trên đặc tính của địa chỉ IP Multicast: Server chỉ cần gửi *một bản duy nhất* của mỗi gói tin tới địa chỉ nhóm; hạ tầng mạng (router/switch hỗ trợ IGMP) sẽ chịu trách nhiệm nhân bản gói tin tới tất cả Client đã đăng ký tham gia nhóm, giúp Server không bị tăng tải khi số lượng Client tăng lên.



Hình 2.1: Kiến trúc truyền video theo mô hình IP Multicast One-to-Many

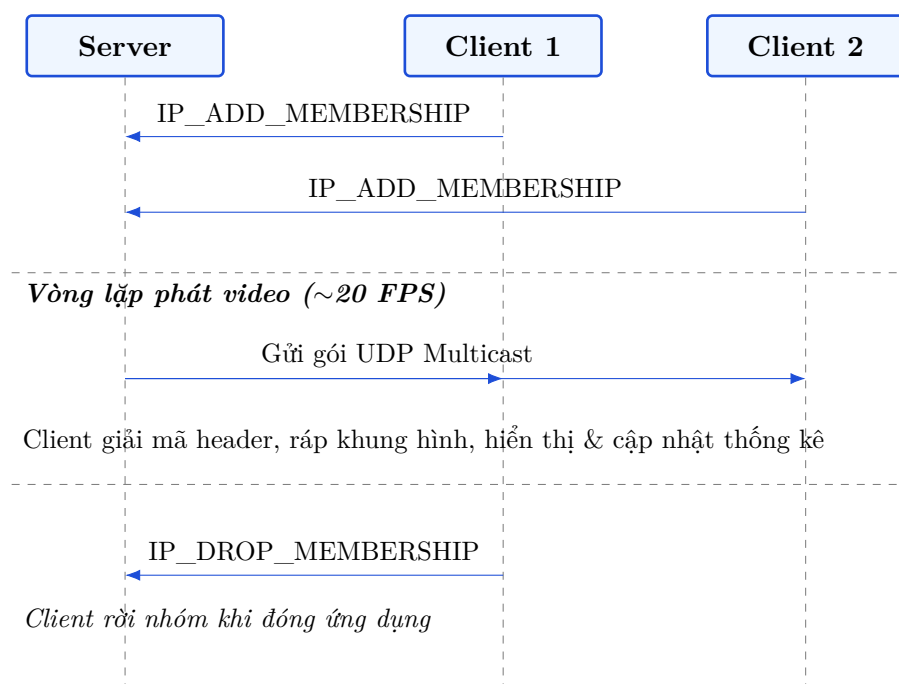
2.2 Thông số Multicast sử dụng

Thông số	Giá trị
Địa chỉ IP Multicast	239.1.1.1
Cổng (Port)	5004
Giao thức tầng vận chuyển	UDP
Multicast TTL (Server)	1 (giới hạn trong LAN)

Địa chỉ 239.1.1.1 thuộc dải **Administrative-Scoped Multicast Address** (239.0.0.0 – 239.255.255.255), được IANA khuyến nghị dùng cho các ứng dụng multicast nội bộ, không định tuyến ra Internet công cộng. Giao thức UDP được lựa chọn vì đặc tính truyền nhanh, không bắt tay (handshake) và không thiết lập kết nối, phù hợp cho luồng video thời gian thực.

2.3 Luồng hoạt động

Trình tự xử lý giữa Server và các Client được minh họa như Hình 2.2.



Hình 2.2: Sơ đồ trình tự hoạt động giữa Server và các Client

Chương 3

Định dạng gói tin tùy biến (Custom Packet Format)

3.1 Vấn đề phân mảnh khung hình

Kích thước tối đa của payload UDP trên thực tế khoảng 65507 bytes. Với các video có độ phân giải cao, một khung hình JPEG có thể vượt quá giới hạn này, gây lỗi phân mảnh gói tin ở tầng hệ điều hành (ví dụ lỗi `WinError 10040` trên Windows). Để giải quyết triệt để vấn đề này, mỗi khung hình JPEG được chia nhỏ thành nhiều **mảnh (fragment)** với kích thước tối đa 60 000 bytes mỗi mảnh trước khi gửi đi, và được Client ráp lại hoàn chỉnh dựa trên thông tin trong header.

3.2 Cấu trúc Header (24 bytes)

Mỗi gói tin UDP gửi đi gồm phần **Header 24 bytes** được đóng gói nhị phân, theo sau là phần **Payload** chứa dữ liệu phân mảnh của ảnh JPEG.

Trường dữ liệu	Kiểu dữ liệu	Kích thước	Struct	Ý nghĩa
Sequence Number	Unsigned Int	4 bytes	I	Phát hiện mất gói
Timestamp	Double Float	8 bytes	d	Tính độ trễ truyền dẫn
Frame Number	Unsigned Int	4 bytes	I	Chỉ số khung hình video
Fragment Index	Unsigned Short	2 bytes	H	Chỉ số mảnh hiện tại
Total Fragments	Unsigned Short	2 bytes	H	Tổng số mảnh khung hình

Payload Length	Unsigned Int	4 bytes	I	Kích thước dữ liệu ảnh
----------------	--------------	---------	---	------------------------

Header được mã hóa theo **Network Byte Order (Big-Endian)** bằng thư viện `struct` của Python, với chuỗi định dạng `"!IdIHHi"` – tổng kích thước cố định 24 bytes.

Listing 3.1: Đóng gói header tại Server (trích `Server.py`)

```

1 header = struct.pack(
2     "!IdIHHi",
3     seq_num,          # Số thứ tự gói tin
4     timestamp,        # Thời điểm gửi
5     frame_num,        # Chỉ số khung hình
6     frag_idx,         # Chỉ số ảnh hiện tại
7     total_fragments,  # Tổng số ảnh
8     len(frag_data)    # Kích thước payload
9 )
10 packet = header + frag_data
11 server_socket.sendto(packet, (MULTICAST_GROUP, PORT))

```

Listing 3.2: Giải mã header và ráp mảnh tại Client (trích `Client.py`)

```

1 header_size = struct.calcsize("!IdIHHi")
2 header, payload = packet[:header_size], packet[header_size:]
3
4 seq_num, timestamp, frame_num, frag_idx, \
5     total_fragments, payload_length = struct.unpack("!IdIHHi", header)
6
7 # Save the fragment to the dictionary using frame_num
8 active_frames[frame_num]['fragments'][frag_idx] = payload[:payload_length]
9
10 # Once all the pieces are collected -> assemble them into a complete JPEG image.
11 if len(frame_info['fragments']) == frame_info['total_fragments']:
12     assembled_jpeg = b"".join(
13         frame_info['fragments'][i] for i in range(total_fragments)
14     )

```

Chương 4

Triển khai chi tiết

4.1 Server (Server.py)

4.1.1 Đọc file MJPEG – lớp MJPEGReader

Server sử dụng lớp MJPEGReader tự xây dựng, hỗ trợ tự động nhận diện 2 định dạng file MJPEG:

- **Định dạng Kurose-Ross:** mỗi khung hình có 5 byte ASCII đứng trước biểu diễn kích thước ảnh JPEG theo sau.
- **Định dạng Raw MJPEG:** chuỗi các ảnh JPEG nối tiếp nhau, được nhận diện qua cặp marker SOI (0xFFD8) và EOI (0xFFD9).

4.1.2 Phát lại video tự động (Loop)

Khi con trỏ đọc tới cuối file (EOF), Server tự động gọi lại `seek(0)` để phát tiếp từ khung hình đầu tiên, đảm bảo luồng video không bị gián đoạn.

4.1.3 Kiểm soát tốc độ khung hình (20 FPS)

Server tính thời gian xử lý mỗi khung hình và chủ động gọi `time.sleep()` phần thời gian còn thiếu để đảm bảo chu kỳ gửi xấp xỉ 50 ms/khung hình (20 FPS):

Listing 4.1: Kiểm soát nhịp phát khung hình

```
1 target_fps = 20
2 frame_interval = 1.0 / target_fps # 0.050s
3
4 frame_elapsed = time.time() - frame_start
5 sleep_time = frame_interval - frame_elapsed
6 if sleep_time > 0:
7     time.sleep(sleep_time)
```

4.1.4 Cấu hình socket Multicast

Server khởi tạo socket UDP (SOCK_DGRAM) và thiết lập IP_MULTICAST_TTL = 1 nhằm giới hạn phạm vi lan truyền gói tin trong mạng LAN nội bộ.

4.2 Client (Client.py)

4.2.1 Giao diện người dùng (GUI)

Client được xây dựng với thư viện `tkinter` kết hợp `Pillow` (PIL) để giải mã và hiển thị ảnh JPEG. Giao diện dạng *Dark Theme* hiện đại, chia làm 2 khu vực:

- **Khung video** (bên trái): hiển thị hình ảnh video, hỗ trợ chế độ toàn màn hình (phím F11).
- **Bảng điều khiển/thống kê** (bên phải): hiển thị các chỉ số Packets Received/Lost, Loss Rate, Framerate, Latency, Bitrate theo thời gian thực.

4.2.2 Xử lý đa luồng (Multi-threading)

Để tránh hiện tượng treo giao diện (GUI freezing), Client tách biệt 2 luồng xử lý:

- **Network Thread** (nền): liên tục gọi `recvfrom()` trên socket UDP Multicast để nhận gói tin.
- **GUI Thread** (chính): định kỳ (mỗi 10 ms) lấy khung hình mới nhất từ hàng đợi `queue.Queue()` để hiển thị.

4.2.3 Ráp lại khung hình từ các mảnh (Reassembly)

Client lưu các mảnh nhận được vào từ điển `active_frames` theo khóa `frame_num`. Khi số mảnh đã nhận bằng `total_fragments` khai báo trong header, Client nối các mảnh theo đúng thứ tự `fragment_index`. Các khung hình quá cũ sẽ được dọn dẹp khỏi bộ nhớ để tránh rò rỉ (memory leak).

4.2.4 Tham gia và rời nhóm Multicast

- **Tham gia nhóm**: sử dụng socket option `IP_ADD_MEMBERSHIP` với cấu trúc `mreq` đóng gói địa chỉ nhóm Multicast.
- **Rời nhóm**: khi người dùng đóng cửa sổ, Client gọi `IP_DROP_MEMBERSHIP` để thông báo Router/Switch ngừng chuyển tiếp luồng Multicast.

4.2.5 Cơ chế phát hiện trạng thái luồng (Online/Offline)

Nếu không nhận được gói tin nào trong hơn 2 giây, giao diện tự động chuyển trạng thái sang **“WAITING FOR STREAM / OFFLINE”**. Khi Server phát trở lại, Client tự động chuyển sang **“STREAMING ACTIVE”**.

Chương 5

Thuật toán đo lường thống kê và phát hiện mất gói

5.1 Phát hiện và tính tỉ lệ mất gói (Packet Loss)

Client lưu biến `last_seq_num` là số thứ tự của gói tin hợp lệ gần nhất. Khi nhận gói mới có `seq_num`:

- Nếu `seq_num > last_seq_num + 1`: số gói bị mất được tính bằng `lost = seq_num - last_seq_num - 1`.
- Tỉ lệ mất gói được tính theo công thức:

$$\text{Tỉ lệ mất gói (\%)} = \frac{\text{Số gói mất}}{\text{Số gói nhận} + \text{Số gói mất}} \times 100\% \quad (5.1)$$

5.2 Đo tốc độ khung hình thực tế (Actual FPS)

Client đếm số gói tin nhận được trong khoảng thời gian 1 giây gần nhất (`interval_packets`) để suy ra tốc độ khung hình thực tế hiển thị trên bảng thống kê.

5.3 Đo băng thông tiêu thụ (Bitrate)

Băng thông được tính bằng tổng kích thước (bits) của các gói tin nhận được trong 1 giây, quy đổi sang đơn vị kbps:

$$\text{Bitrate (kbps)} = \frac{\text{Tổng bytes nhận trong 1s} \times 8}{1024} \quad (5.2)$$

5.4 Đo độ trễ truyền dẫn (Estimated Latency)

Độ trễ được ước lượng bằng cách so sánh mốc thời gian nhận gói tin hiện tại tại Client với trường `Timestamp` lưu trong header:

$$\text{Latency (ms)} = (t_{\text{nhận}} - t_{\text{gửi (header)}}) \times 1000 \quad (5.3)$$

Chương 6

Hướng dẫn cài đặt và chạy chương trình

6.1 Yêu cầu môi trường

Chương trình yêu cầu Python 3 và thư viện Pillow:

Listing 6.1: Cài đặt thư viện cần thiết

```
pip install Pillow
```

Để kiểm thử chương trình, người dùng cần chuẩn bị một tệp video định dạng `.mjpeg`. Có thể tải một video MJPEG từ Internet hoặc sử dụng tệp được nhóm cung cấp, sau đó chép tệp vào cùng thư mục với chương trình `Server.py`.

6.2 Khởi chạy Server

Listing 6.2: Chạy Server phát video

```
python Server.py movie.mjpeg
```

6.3 Khởi chạy Client

Listing 6.3: Chạy Client nhận và xem video

```
python Client.py
```

Chương 7

Kết quả kiểm thử

7.1 Kịch bản kiểm thử

softblue STT	Kịch bản	Kết quả quan sát
1	Chạy 1 Server và 3 Client trên cùng máy	Các Client hiển thị hình ảnh đồng bộ với nhau.
2	Đo độ trễ truyền phát trên localhost	Độ trễ cực thấp, dưới 1 ms.
3	Đo tỉ lệ mất gói trên localhost	Duy trì ổn định ở mức 0.00%.
4	Nhấn nút <i>Reset Stats</i>	Số liệu thống kê được xóa và tính lại chính xác.
5	Đóng cửa sổ Client	Client rời nhóm và giải phóng socket nhanh chóng.
6	Video phát hết khung hình cuối	Server tự động phát lại từ đầu.

7.2 Nhận xét

Do môi trường kiểm thử chạy trên cùng một máy (localhost), độ trễ và tỉ lệ mất gói gần như bằng 0. Nhóm khuyến nghị triển khai thử nghiệm trên các máy khác nhau kết nối qua mạng Wi-Fi/LAN để đánh giá rõ ràng hơn.

7.3 Đối chiếu với tiêu chí chấm điểm

softblue Tiêu chí	Điểm	Minh chứng đáp ứng
-------------------	------	--------------------

Cài đặt Server	2.5	<code>Server.py</code> – phát Multicast 20 FPS, tự lặp video.
Cài đặt Client	2.5	<code>Client.py</code> – tham gia nhóm, nhận, hiển thị video.
Định dạng gói tin	2.0	Header nhị phân 24 bytes tùy biến, có phân mảnh.
Nhiều Client & phát hiện mất gói	2.0	Kiểm thử đồng thời nhiều Client; tính tỉ lệ mất gói.
Báo cáo	1.0	Kiến trúc và quá trình kiểm thử được trình bày đầy đủ.

Chương 8

Kết luận và hướng phát triển

8.1 Kết luận

Đồ án đã hoàn thành đầy đủ các yêu cầu chức năng đề ra: xây dựng thành công hệ thống truyền phát video thời gian thực qua giao thức IP Multicast trên nền UDP. Nhóm đã tự thiết kế định dạng gói tin nhị phân tùy biến (24 bytes header) có khả năng phân mảnh/ráp mảnh để xử lý các khung hình có kích thước lớn, đáp ứng đầy đủ các tiêu chí của đề bài.

8.2 Hướng phát triển

- Bổ sung cơ chế mã hóa video hiệu quả hơn (ví dụ H.264) thay vì MJPEG để giảm băng thông tiêu thụ.
- Thử nghiệm và tối ưu hệ thống trên môi trường mạng thực tế (nhiều máy khác nhau).
- Nghiên cứu bổ sung cơ chế điều chỉnh chất lượng video thích ứng (adaptive bitrate).

Tài liệu tham khảo

- [1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, Pearson, 7th Edition.
- [2] Python Software Foundation, *socket — Low-level networking interface*, Python 3 Documentation. <https://docs.python.org/3/library/socket.html>
- [3] Nguyễn Trọng Phúc, Phạm Minh Quân, *Project02 – Multicast Video Streaming Source Code*, GitHub Repository. <https://github.com/Chintaro05/Project02>